Universidad Peruana de Ciencias Aplicadas



HITO 2 SEGUNDO AVANCE

StockLang

Lenguaje para describir inventarios

CURSO TEORÍA DE COMPILADORES

Carrera de Ciencias de la Computación

Sección: 4889

Profesor: JOSE LUIS SONCCO ALVAREZ

Nombres y apellidos	
U202223356	Antonio Morán, José David
U202321420	Cordova Montenegro, Sebastian

2026-2

1 Problemática y motivación

Las pequeñas tiendas necesitan registrar altas, reposiciones y retiros de productos con reglas claras. StockLang expresa estas operaciones en español y comprueba productos inexistentes, cantidades inválidas y resultados incoherentes antes de ejecutar el inventario.

2 Objetivos

Construir un compilador con ANTLR4 que reconozca programas de inventario y compruebe sus reglas. En este avance incorporamos sino, completamos siete controles semánticos, documentamos la arquitectura y contrastamos los resultados con una máquina virtual.

3 Construcciones y ejemplos

Construcción y propósito	**Cinco ejemplos por construcción**
Declaración Define el producto y su stock inicial.	producto lapiz stock 100; producto cuaderno stock 25; producto borrador stock 0; producto regla stock 8; producto mochila stock 5;
Movimiento Expresa una entrada o salida de unidades.	entrada lapiz 10; salida cuaderno 3; entrada borrador 2 * 5; salida regla 1 + 1; entrada mochila (3 + 2) * 2;

Ajuste
Asigna una expresión al stock del producto.
	ajustar lapiz = 80;
	ajustar cuaderno = cuaderno + 5;
	ajustar borrador = 4 * 6;
	ajustar regla = regla - 1;
	ajustar mochila = (2 + 3) * 2;

Consulta Solicita mostrar un producto.	mostrar lapiz; mostrar cuaderno; mostrar borrador; mostrar regla; mostrar mochila;
Condicional Agrupa instrucciones bajo una comparación.	si lapiz < 10 { entrada lapiz 20;} sino { mostrar lapiz;} si cuaderno == 0 { entrada cuaderno 5;} sino { mostrar cuaderno;} si borrador > 50 { salida borrador 5;} sino { entrada borrador 5;} si regla <= 3 { ajustar regla = 10;} sino { mostrar regla;} si mochila >= 5 { mostrar mochila;salida mochila 1;} sino { entrada mochila 2;}
Expresión Combina enteros y referencias a stock.	10 lapiz + 5 2 * 3 + 1 cuaderno - 2 (4 + 6) * 2

Cada ejemplo se prueba por separado. Para instrucciones se usan lapiz=100, cuaderno=25, borrador=60, regla=8 y mochila=5. Las expresiones se insertan en un ajuste. representa el terminador obligatorio de cada instrucción en la notación impresa de este informe.

4 Léxico y modelo de datos

El stock es entero y pertenece al rango de 0 a 2147483647. Los identificadores distinguen mayúsculas y minúsculas. Las expresiones permiten valores intermedios entre -2147483648 y 2147483647. No existe signo unario ni división. El lexer conserva línea y columna y omite espacios y comentarios.

Familia	Token y lexema o patrón
Reservadas	PRODUCTO: producto, STOCK: stock, ENTRADA: entrada, SALIDA: salida, AJUSTAR: ajustar, MOSTRAR: mostrar, SI: si, SINO: sino.
Comparación	MENOR: <, MENOR_IGUAL: <=, MAYOR: >, MAYOR_IGUAL: >=, IGUAL: ==, DISTINTO: !=.
Operadores	ASIGNAR: =, SUMA: +, RESTA: -, MULTIPLICAR: *.
Delimitadores	; : cierre de sentencia LLAVE_IZQ: { LLAVE_DER: } PAREN_IZQ: (PAREN_DER:).
Categorías	ID: [a-zA-Z_][a-zA-Z_0-9]*, como lapiz o item_2. NUMERO: [0-9]+, como 0, 25 o 007.
Omitidos y fin	COMENTARIO: // hasta fin de línea. ESPACIO: espacios, tabulaciones y saltos de línea. EOF indica fin de entrada.

Se aplica la coincidencia más larga. En empate, las reservadas preceden a ID. Un carácter sin regla produce L001. Las columnas se presentan desde 1.

5 Gramática libre de contexto

Programa → Declaración ListaDeclaraciones ListaInstrucciones EOF
ListaDeclaraciones → Declaración ListaDeclaraciones | ε
ListaInstrucciones → Instrucción ListaInstrucciones | ε
Instrucción → Movimiento | Ajuste | Consulta | Condicional
Declaración → producto identificador stock número;
Movimiento → TipoMovimiento identificador Expresión;
TipoMovimiento → entrada | salida
Ajuste → ajustar identificador = Expresión;
Consulta → mostrar identificador;
Condicional → si Condición Bloque Alternativa
Alternativa → sino Bloque | ε
Condición → Expresión Comparador Expresión
Comparador → < | <= | > | >= | == | !=
Bloque → { Instrucción ListaInstrucciones }
Expresión → Expresión + Término | Expresión - Término | Término
Término → Término * Factor | Factor
Factor → número | identificador | (Expresión)
La flecha indica producción, la barra una alternativa y ε la cadena vacía. identificador y número son categorías terminales de ID y NUMERO. Las declaraciones van primero. Los bloques contienen al menos una instrucción. La jerarquía Expresión, Término y Factor establece precedencia y asociación izquierda.

En ANTLR, condicional reconoce SI, una condición y un bloque, seguidos opcionalmente por SINO y otro bloque. Se conserva si sin alternativa y se admiten decisiones anidadas delimitadas por llaves.

6 Derivaciones por la izquierda

Las 24 derivaciones completas se incluyen en el anexo adjunto, cuatro por cada construcción. Cada paso indica la producción aplicada y la cadena resultante. Se utilizan nombres como Expresión, Condición y Bloque. La negrita identifica el siguiente no terminal que se debe expandir y ; representa el cierre de una instrucción.

7 Implementación y semántica completa

SemanticAnalyzer mantiene una tabla global de símbolos con nombre, stock inicial y línea de declaración, y un mapa separado con el stock calculado. Revisa destinos, expresiones, comparaciones y bloques anidados. Se implementaron las siguientes siete clases de error:

Código	Regla comprobada	Programa inválido representativo
E001	Producto duplicado	producto a stock 1; producto a stock 2;
E002	Producto no declarado	producto a stock 1; mostrar b;
E003	Stock inicial fuera de rango	producto a stock 2147483648;
E004	Movimiento no positivo	producto a stock 10; entrada a 0;
E005	Ajuste negativo	producto a stock 10; ajustar a = 1 - 2;
E006	Stock insuficiente	producto a stock 10; salida a 11;
E007	Desbordamiento entero	producto a stock 2147483647; entrada a 1;

La primera declaración se conserva ante duplicados. Una operación inválida mantiene el último stock válido. Una expresión errónea devuelve un valor desconocido para evitar diagnósticos numéricos derivados. L001 y S001 detienen el acceso a fases posteriores. Cualquier error semántico impide generar IR y ejecutar el programa.

Comportamiento del condicional

La condición se evalúa con el estado actual. El bloque si se comprueba sobre una copia del inventario. Luego se restaura el estado de entrada y se comprueba sino, si existe. Solo se propaga el estado de la rama elegida. Sin sino y con condición falsa se conserva el inventario anterior.

```
producto a stock 10;
si a < 0 { ajustar a = 100;} sino { salida a 9;}
mostrar a;
```

Resultado verificado: a = 1. La actualización de la rama inactiva no modifica el stock con el que se analiza sino. También se probaron condiciones verdaderas, falsas y anidadas, errores dentro de sino y rechazo de bloques vacíos.

Política conservadora. Ambas ramas deben ser válidas incluso cuando una no se ejecuta. Por ello, con stock 0, un retiro de 1 dentro de si a > 0 se rechaza con E006. Es una decisión implementada y probada que limita las operaciones protegidas por condiciones. Su revisión queda pendiente.

8 Arquitectura del compilador

La arquitectura separa reconocimiento, comprobación del inventario y ejecución. driver.py recibe el archivo y src/compiler.py coordina las fases. Cada componente entrega un resultado definido al siguiente. Un diagnóstico bloquea el avance correspondiente.

Componente	Entrada	Salida y responsabilidad
StockLangLexer	Caracteres UTF 8	Tokens, lexemas y posiciones. Emite L001.
StockLangParser	Flujo de tokens	Árbol sintáctico hasta EOF. Emite S001.
SemanticAnalyzer	Árbol sintáctico	Tabla de símbolos, stock verificado y E001 a E007.
IRGenerator	Árbol sin errores	Instrucciones de tres direcciones en objetos JSON.
execute	Código intermedio	Inventario final y salida de mostrar.
AnalysisResult	Resultados de las fases	tokens, tree, symbols, diagnostics, final_stocks, ir y execution.

Flujo y contratos

Archivo StockLang → lexer → parser → visitante semántico → generador IR → máquina virtual. Los campos symbols conservan el stock declarado. final_stocks contiene el estado calculado por el análisis. execution registra el resultado independiente de la VM. La máquina virtual no recibe archivos IR externos.

El backend emplea declare, const, load, add, sub, mul, positive, store, cmp, branch_false, jump, label y print. Los temporales y las etiquetas se numeran de forma determinista. El condicional usa comparaciones y saltos para ejecutar únicamente su alternativa seleccionada.

Ejemplo de código intermedio

Para entrada lapiz 2 * 3; se generan operaciones equivalentes a las siguientes

```
const t1, 2
const t2, 3
mul t3, t1, t2
load t4, lapiz
positive t3
add t5, t4, t3
store lapiz, t5
```

Esta notación facilita la lectura. El archivo real contiene objetos JSON. La VM comprueba cantidades positivas, no negatividad del stock y límites enteros durante las operaciones.

Interfaz y alcance del avance

La CLI permite --stage lex/syntax/semantic, --tokens, --tree, --json, --emit-ir y --run. Retorna 0 si acepta, 1 si rechaza el programa y 2 ante argumentos o archivos inválidos. La versión usa Python 3.10 o superior y antlr4-python3-runtime 4.13.2. Los reconocedores generados permiten ejecutar sin Java.

9 Plan de validación

El plan combina aceptación y rechazo por fase, fronteras numéricas, regresión del Hito 1 y flujo por ramas. Los criterios son cero fallos, diagnóstico y posición correctos, ausencia de IR en entradas inválidas y coincidencia entre stock verificado y ejecución. La demo usa un cálculo manual, 80 programas de movimientos usan un contador independiente y semilla 20261110.

10 Pruebas y resultados

Indicador observado	Resultado verificado
Métodos unittest	55 aprobados, 0 fallos y 0 errores.
Consola	13 de 13 casos con retorno y diagnóstico esperados.
Construcciones y derivaciones	30 muestras aceptadas y 24 derivaciones izquierdas verificadas.
Oráculo independiente	80 programas dentro de un método unittest. No son 80 métodos adicionales.
Demo	lapiz = 15 y cuaderno = 5. Coinciden semántica y VM.

Caso de examples	Diagnóstico esperado	Retorno observado
tokens.stock / --stage lex	Aceptado. Tokens y EOF	0
valido.stock	Aceptado	0
demo.stock / --run	Aceptado. lapiz=15, cuaderno=5	0
error_lexico.stock	L001	1
error_sintactico.stock	S001	1
error_duplicado.stock	E001	1
error_no_declarado.stock	E002	1
error_condicional.stock	E002	1
error_stock_inicial.stock	E003	1
error_cantidad.stock	E004	1
error_ajuste_negativo.stock	E005	1
error_insuficiente.stock	E006	1
error_desbordamiento.stock	E007	1

Límites y seguimiento. Un literal de 4301 dígitos produce ValueError de Python y requiere un diagnóstico controlado.

URL de referencia: https://github.com/fenix800/StockLang-Hito1

Referencias. Soncco Álvarez, J. L. (2026). Teoría de Compiladores Trabajo Parcial y Final 2026 2 [Enunciado del curso]. Equipo StockLang (2026). Código e informe del Hito 1 y paquete del Hito 2. ANTLR Project (s. f.). Python target, versión 4.13.2. https://github.com/antlr/antlr4/blob/4.13.2/doc/python-target.md